Statistical Tools for Data Scientists

This notebook demonstrates practical statistical tools used in real data science workflows.

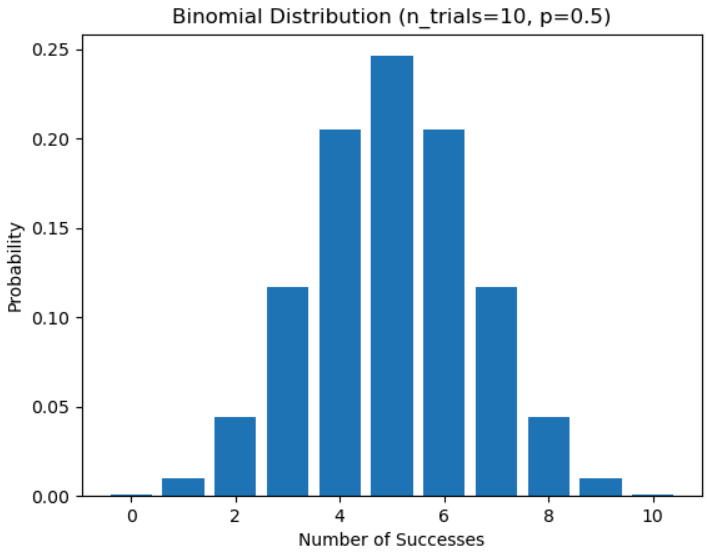
In [1]:
```
!pip install scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
```

```
Requirement already satisfied: scipy in /opt/conda/lib/python3.12/site-packa
ges (1.17.1)
Requirement already satisfied: numpy<2.7,>=1.26.4 in /opt/conda/lib/python3.
12/site-packages (from scipy) (1.26.4)
```

Binomial Distribution

A binomial distribution is used when:

- There are fixed number of trials
- Each trial has two outcomes
- Each trial has same probability of success
- Trials are independent



Example

Email marketing conversions

A company sends marketing emails to customers. Each email can result in two outcomes:

- Customer converts (clicks + buys)
- Customer does not convert

This is a Bernoulli trial. If we send multiple emails, the number of conversions follows a Binomial distribution.

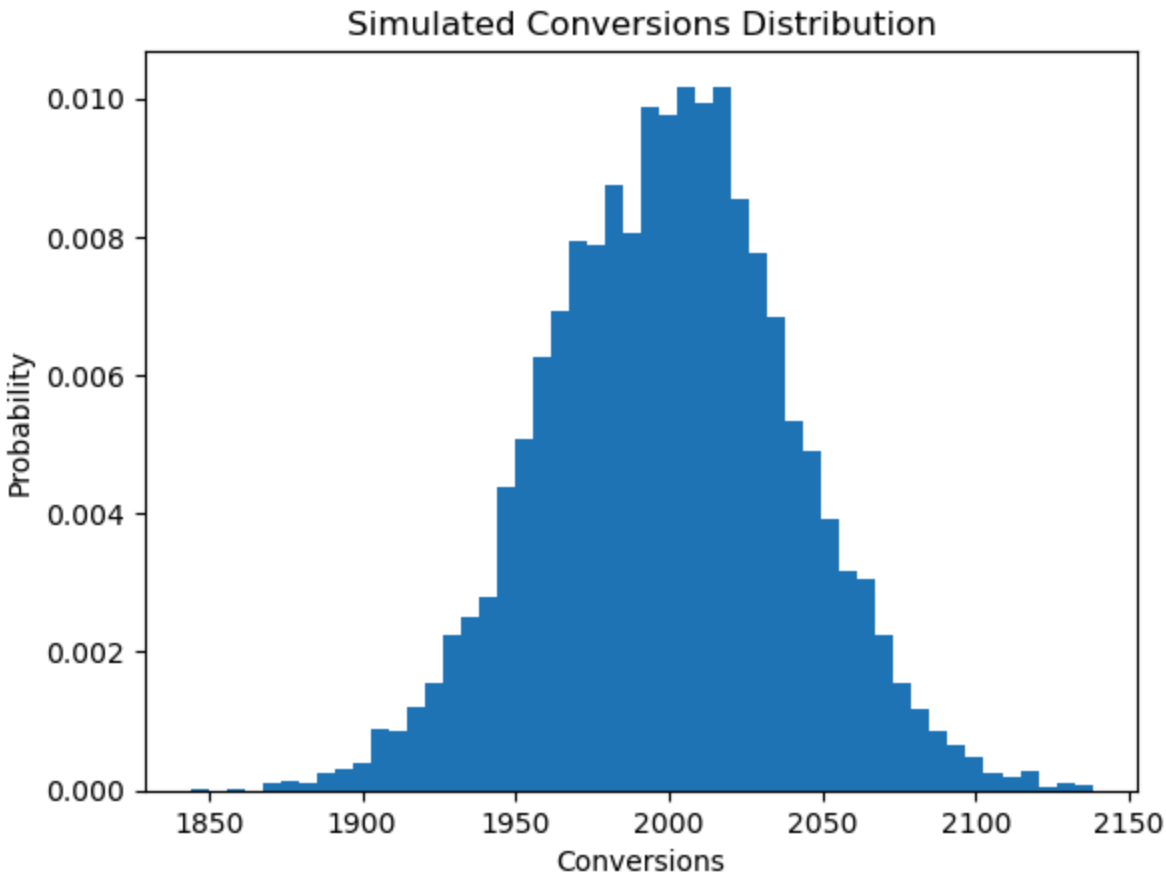
Suppose:

- Conversion probability = 0.2 (20%)
- Emails sent = 10000 We want to know:
- Expected conversions
- Probability of 2000 conversions
- Simulated outcomes

In [2]:
```
n_trials = 10000
probability = 0.2
binom_dist = stats.binom(n_trials, probability)
```

In [3]:
```
# simulate conversions
# This simulates running the same marketing campaign multiple times.
simulated_data=binom_dist.rvs(size=10000)   # Each number represents how mar
```

In [4]:
```
# Plot simulation histogram
plt.hist(simulated_data, bins=50, density=True)
plt.xlabel('Conversions')
plt.ylabel('Probability')
plt.title('Simulated Conversions Distribution');
```



In [6]:
```
# Probability of exactly 2000 conversions
binom_dist.pmf(2000) # If we run one campaign, what is the probability that
```

Out[6]: 0.009973120676464978

In [7]:
```
# What's the risk that nobody buys anything in this campaign?
binom_dist.pmf(0)
```

Out[7]: 0.0

In [8]:
```
# Probability that 2000 or fewer customers convert (click + buy) in this cam
answer=binom_dist.cdf(2000)
print(f"{answer:.5f}")
```

```
0.50598
```

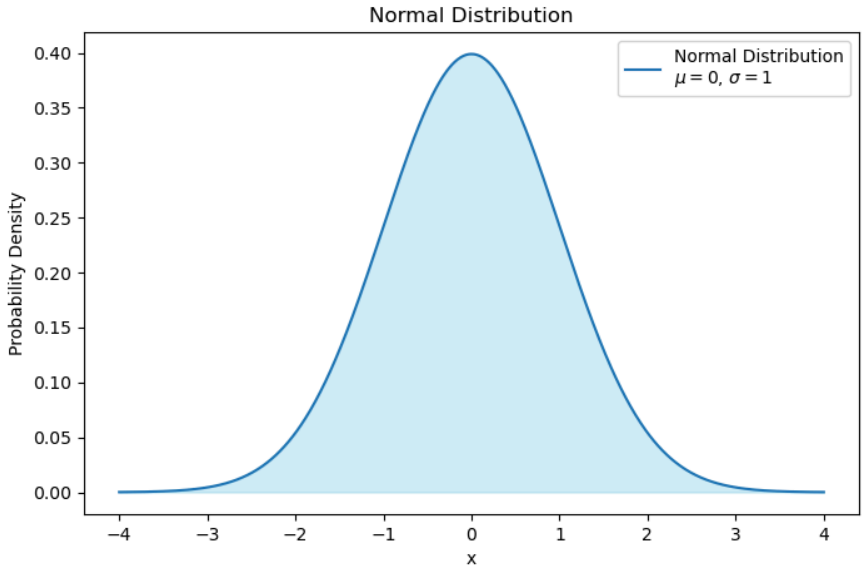
Normal Distribution

Most performance metrics follow a Normal distribution.

Examples:

- website response time
- delivery time
- product weight

- measurement errors



Example

A company measures website page load time.

Example values:

```
1.8 seconds
2.1 seconds
1.9 seconds
2.4 seconds
1.7 seconds
```

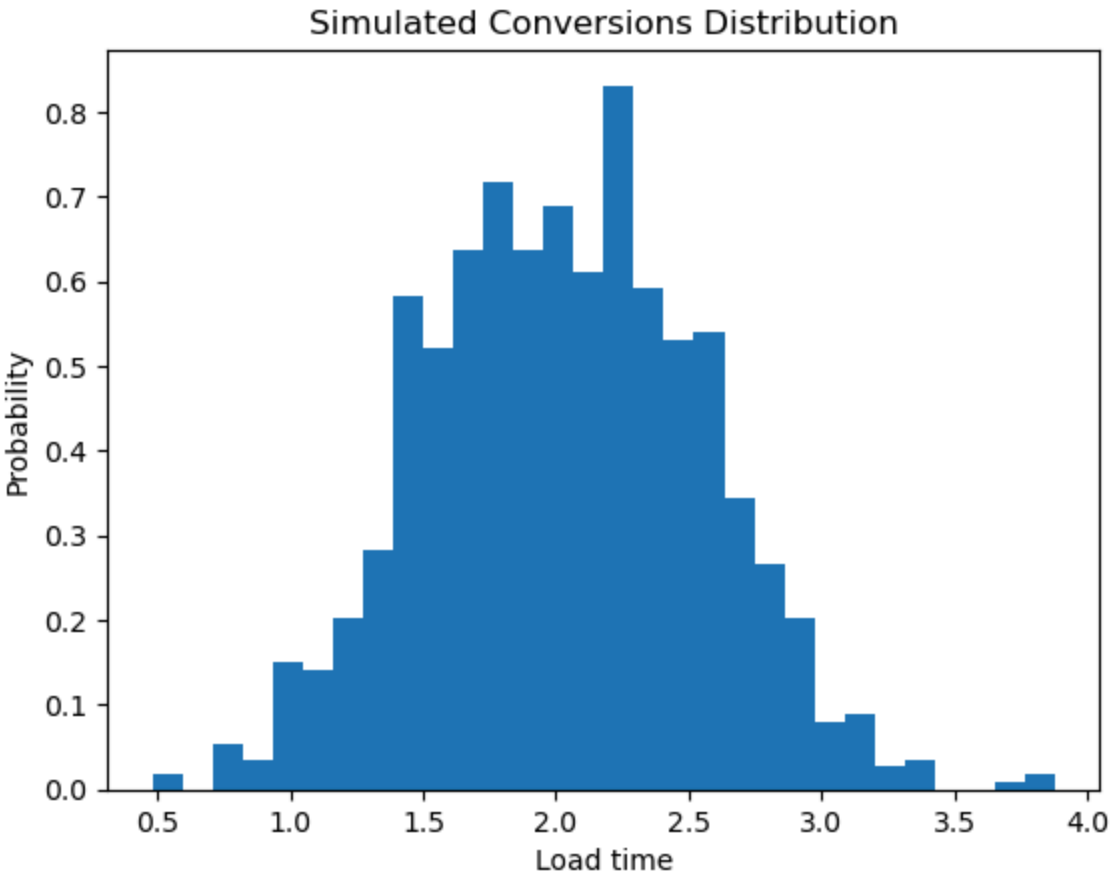
- Average load time = 2 seconds
- with Small variation due to:
  - network
  - server load
  - user device

In [9]:
```
# Example: website response time
mean = 2
variance = 0.5
normal_dist = stats.norm(mean, variance)
```

In [10]:
```
# This generates 1000 simulated page loads.
load_times = normal_dist.rvs(size=1000)
```

In [11]:
```
# Plot simulation histogram
plt.hist(load_times, bins=30, density=True);
plt.xlabel('Load time')
```

```
plt.ylabel('Probability')
plt.title('Simulated Conversions Distribution');
```



In [12]:
```
# probability density
normal_dist.pdf(2)   # the likelihood density around 2 seconds (results show
```

Out[12]:
```
0.7978845608028654
```

In [13]:
```
# cumulative probability
# Probability load time ≤ 2 seconds
normal_dist.cdf(2)   # What fraction of users experience load time ≤ 2 secon
```

Out[13]:
```
0.5
```

Distribution Diagnostics

In [14]:
```
# Normality test
# Tests the null hypothesis (H0) that a sample comes from a normal distribut
# If the p-value < alpha, we reject H0, indicating the sample likely does NO

stats.normaltest(load_times)
```

Out[14]:
```
NormaltestResult(statistic=1.7710706801387106, pvalue=0.4124932899740197)
```

Skewness Test

In [15]:
```
# Null hypothesis (H0): The data is symmetric (skew = 0)
#  p < alpha (e.g., 0.05)   → reject H0          → data is significantly ske
#  p >= alpha               → fail to reject H0  → data is not significantly
stats.skewtest(load_times)
```

Out[15]:
```
SkewtestResult(statistic=1.1017784227962832, pvalue=0.27055801317311357)
```

kurtosis Test

In [16]:
```
# Null hypothesis (H0): The data has normal kurtosis (i.e., kurtosis = 3)
#  p < alpha (e.g., 0.05)   → reject H0          → data has significantly no
#  p >= alpha               → fail to reject H0  → data does not significant
stats.kurtosistest(load_times)
```

Out[16]:
```
KurtosistestResult(statistic=-0.746428152737586, pvalue=0.455408818024823)
```

One Sample t-test

- Tests whether the mean of a sample is equal to a known value (population mean μ0)
  - Null hypothesis (H0): sample mean = popmean
  - Alternative hypothesis (H1): sample mean ≠ popmean
- Interpretation:
  - If p < alpha (e.g., 0.05) → reject H0 → sample mean is significantly different from popmean
  - If p >= alpha → fail to reject H0 → no significant difference from popmean

In [17]:
```
# H0: mean load time = 2 seconds
stats.ttest_1samp(load_times, 2)
```

Out[17]:
```
TtestResult(statistic=1.6359098224134532, pvalue=0.10217345354226272, df=99
9)
```

Independent Two Sample t-test

- Tests whether the means of two independent samples are equal
  - Null hypothesis (H0): the two samples have equal means
  - Alternative hypothesis (H1): the two samples have different means
- Interpretation:
  - If p < alpha (e.g., 0.05) → reject H0 → means are significantly different
  - If p >= alpha → fail to reject H0 → no significant difference between means

In [19]:
```
version_A = stats.norm(2,0.5).rvs(size=100)
version_B = stats.norm(2.2,0.5).rvs(size=100)

stats.ttest_ind(version_A, version_B)
```

Out[19]:
```
TtestResult(statistic=-2.570151769667541, pvalue=0.010900059130798279, df=198.0)
```

In [20]:
```
# Example - Sanity Check via Resampling
sub_sample1 = np.random.choice(load_times, size=20, replace=True)
sub_sample2 = np.random.choice(load_times, size=20, replace=True)

stats.ttest_ind(sub_sample1, sub_sample2)
```

Out[20]:
```
TtestResult(statistic=1.2292835064282808, pvalue=0.2265232393104371, df=38.0)
```

One-way ANOVA (Analysis of Variance)

- Tests whether the means of two or more independent groups are equal
  - Null hypothesis (H0): all group means are equal
  - Alternative hypothesis (H1): at least one group mean is different
- Interpretation:
  - If p < alpha (e.g., 0.05) → reject H0 → at least one group differs significantly
  - If p >= alpha → fail to reject H0 → no significant difference among group means

In [21]:
```
campaign1 = stats.norm(50,5).rvs(size=50)
campaign2 = stats.norm(55,5).rvs(size=50)
campaign3 = stats.norm(60,5).rvs(size=50)

stats.f_oneway(campaign1, campaign2, campaign3)
```

Out[21]:
```
F_onewayResult(statistic=77.24902784655278, pvalue=1.1760131902342904e-23)
```

Example

- Compare test scores of students taught using three different teaching methods.
  - H0: All group means are equal
  - H1: At least one mean differs

In [22]:
```
# Sample data: three groups
group_A = [23, 21, 25, 22]
group_B = [30, 28, 27, 29]
group_C = [20, 19, 21, 22]
```

```
stats.f_oneway(group_A, group_B, group_C)
```

Out[22]:
```
F_onewayResult(statistic=32.67999999999995, pvalue=7.46558900540892e-05)
```

Chi-Square Goodness of Fit

- Tests whether the observed frequencies match the expected frequencies
  - Null hypothesis (H0): observed frequencies = expected frequencies
  - Alternative hypothesis (H1): observed frequencies ≠ expected frequencies
- Interpretation:
  - If p < alpha (e.g., 0.05) → reject H0 → observed counts significantly differ from expected
  - If p >= alpha → fail to reject H0 → no significant difference

In [23]:
```
observed = [30,25,20,25]
expected = [25,25,25,25]

stats.chisquare(f_obs=observed, f_exp=expected)
```

Out[23]:
```
Power_divergenceResult(statistic=2.0, pvalue=0.5724067044708798)
```

Example

- Scenario: A candy company produces M&M's in 5 colors. They claim each color is equally likely. You buy a bag and count the colors: | Color | Red | Blue | Green | Yellow | Orange | Brown | | -------- | --- | ---- | ----- | ------ | ------ | ----- | | Observed | 12 | 10 | 8 | 15 | 9 | 6 |
- **Question:** Do the observed counts match the expected uniform distribution?

In [24]:
```
observed = [12, 10, 8, 15, 9, 6]
stats.chisquare(f_obs=observed)
```

Out[24]:
```
Power_divergenceResult(statistic=5.0, pvalue=0.4158801869955079)
```

Chi-Square Test of Independence

- Tests whether two categorical variables are independent
  - Null hypothesis (H0): the variables are independent
  - Alternative hypothesis (H1): the variables are dependent
- Interpretation:

- If p < alpha (e.g., 0.05) → reject H0 → variables are significantly associated
- If p >= alpha → fail to reject H0 → no significant association

In [ ]:
```
crosstab = np.array([
 [50,30],
 [40,60]
])

stats.chi2_contingency(crosstab)
```

Example

- A health researcher wants to know if smoking status is associated with lung disease. They collect data from 200 people:

	Lung Disease	No Lung Disease	Total
Smoker	40	60	100
Non-Smoker	10	90	100

Question: Are smoking and lung disease independent, or is there a significant association?

In [25]:
```
# Null hypothesis (H0): Smoking and lung disease are independent.
# Alternative hypothesis (H1): Smoking and lung disease are not independent

observed = [[40, 60],   # Smokers
            [10, 90]]   # Non-Smokers

stats.chi2_contingency(observed)
```

Out[25]:
```
Chi2ContingencyResult(statistic=22.42666666666667, pvalue=2.183216533714853
7e-06, dof=1, expected_freq=array([[25., 75.],
       [25., 75.]]))
```

In [ ]:
```

```